

CVM++

Custom Virtual Machine in C++

A Stack-Based Bytecode Compiler & Runtime Engine

Even Semester Project Report

Language: C++17 | Build: CMake 3.15+
Reference: Crafting Interpreters by Robert Nystrom

1. Introduction

CVM++ is a lightweight, fully functional custom scripting language, bytecode compiler, and stack-based Virtual Machine built entirely from scratch in C++17. The project implements a complete end-to-end pipeline that transforms raw, human-readable source code (.cvm files) into proprietary bytecode, which is then executed sequentially by an isolated, deterministic runtime engine.

This project draws inspiration from Part II of Robert Nystrom's "Crafting Interpreters" (the clox bytecode VM), adapted into a modern C++17 codebase with unique design decisions. It was developed to demonstrate mastery of compiler theory, language design, and low-level systems programming.

2. Problem Statement

Design and implement a custom programming language with its own compiler and virtual machine. The system must include: (1) a Lexer that converts raw source into tokens, (2) a Parser that builds an Abstract Syntax Tree using recursive descent, (3) a Compiler that emits proprietary bytecode from the AST, and (4) a stack-based Virtual Machine that executes the bytecode. The language must support variables, arithmetic, comparisons, conditional branching (if/else), iterative loops (while), and standard I/O (print/input).

3. Project Objectives

- Build a four-stage compilation pipeline: Source -> Tokens -> AST -> Bytecode -> Execution.
- Design a formal BNF grammar for a minimalistic yet Turing-complete scripting language.
- Implement a Lexer with full tokenization, keyword recognition, and line-level error reporting.
- Implement a recursive descent Parser that constructs a well-formed AST.

- Define a custom Instruction Set Architecture (ISA) with 22 distinct opcodes.
- Implement a Compiler with backpatching for control flow jumps (if/else and while loops).
- Implement a stack-based VM with an execution loop, global variable storage, and error handling.
- Provide REPL and file-runner CLI modes with extensive debug flags (--tokens, --ast, --bytecode).
- Create 13+ test scripts demonstrating all language features robustly.

4. High-Level Architecture

The architecture follows a classic four-stage compiler pipeline. Each stage is encapsulated in its own C++ module with a clean header/implementation split under the include/ and src/ directories respectively.

Source Code (.cvm)

```
|
+-> [Lexer]      Tokenizes raw characters into Tokens
+-> [Parser]     Builds AST via recursive descent
+-> [Compiler]   Traverses AST, emits bytecode to Chunk
+-> [VM]        Executes bytecode on a stack machine
|
Output
```

5. Language Design

5.1 Formal Grammar (BNF Specification)

The syntax of CVM++ is heavily inspired by modern scripting languages such as JavaScript and C. It aims to be minimalistic yet expressive enough to facilitate complex algorithmic implementations.

```

program    -> statement* EOF
statement  -> letStmt | printStmt | inputStmt
            | ifStmt | whileStmt | exprStmt
letStmt    -> 'let' IDENTIFIER '=' expression ';'
printStmt  -> 'print' expression ';'
inputStmt  -> 'input' IDENTIFIER ';'
ifStmt     -> 'if' '(' expression ')' block ( 'else' block )?
whileStmt  -> 'while' '(' expression ')' block
block      -> '{' statement* '}'
expression -> assignment
assignment -> IDENTIFIER '=' expression | equality
equality   -> comparison ( ('==' | '!=') comparison )*
comparison -> term ( ('<' | '>' | '<=' | '>=') term )*
term       -> factor ( ('+' | '-') factor )*
factor     -> unary ( ('*' | '/') unary )*
unary      -> ('-' | '!') unary | primary
primary    -> NUMBER | 'true' | 'false' | IDENTIFIER | '(' expression ')'

```

5.2 Supported Data Types and Operators

Data Type	Examples	Internal Representation
Integer	0, 42, -7	int64_t
Boolean	true, false	int64_t (1 or 0)

Category	Operators
Arithmetic	+ - * /
Comparison	== != < > <= >=
Logical	! (unary NOT)
Assignment	=

6. Lexical Analysis (The Lexer)

The Lexer reads raw source as a character stream and produces an array of Token structs. Each Token contains a TokenType enum (one of 24 types), the raw lexeme string, and the source line number for precise error reporting. The lexer elegantly handles single-character

tokens, two-character operators using one-character lookahead, numeric literals, and alphanumeric identifiers which are cross-checked against a map of reserved keywords.

7. Parsing (Abstract Syntax Tree)

The Parser implements a top-down Recursive Descent strategy. Each BNF grammar rule maps directly to a dedicated C++ method. The parser consumes the token array produced by the Lexer and constructs an Abstract Syntax Tree (AST).

7.1 AST Node Hierarchy & Memory Management

All nodes inherit from a polymorphic `ASTNode` base class with a virtual `print()` method for debug output. Memory ownership is strictly managed via `std::unique_ptr`, guaranteeing automatic, leak-free memory cleanup upon destruction of the AST root block.

7.2 Operator Precedence

Precedence is enforced structurally by the recursive call chain depth, not by an external lookup table. The parsing descends from assignment (lowest) down to equality, comparison, term, factor, unary, and finally primary (highest). This guarantees that multiplication and division bind tighter than addition and subtraction natively.

8. Instruction Set Architecture (ISA)

The CVM++ ISA defines 22 unique opcodes encoded as `uint8_t` values within a `Chunk` structure. The `Chunk` struct holds parallel arrays: 'code' for the bytecode stream, 'constants' for the `int64_t` literal pool, and 'names' for deduplicated variable identifiers.

Opcode	Operand	Description
OP_CONSTANT	1 byte idx	Push constants[idx] onto stack
OP_POP	none	Discard top of stack
OP_ADD/SUB/MUL	none	Binary arithmetic (inc DIV)

OP_NEGATE	none	Negate top of stack
OP_EQUAL/!=	none	Equality comparison
OP_LESS/>	none	Relational comparison
OP_NOT	none	Logical NOT
OP_TRUE/FALSE	none	Push boolean literal (1/0)
OP_SET_GLOBAL	1 byte idx	Store top to global var
OP_GET_GLOBAL	1 byte idx	Load global var onto stack
OP_JUMP	2 byte offset	Unconditional branch
OP_JUMP_IF_FALSE	2 byte offset	Conditional branch
OP_PRINT/INPUT	none	Standard I/O operations
OP_HALT	none	Terminate execution

9. Compilation (Bytecode Generation)

The Compiler performs a post-order traversal of the AST and emits linear bytecode into a Chunk. It leverages `dynamic_cast` to safely downcast and dispatch each polymorphic `ASTNode` to its appropriate compilation subroutine.

9.1 Backpatching for Control Flow

Forward jumps (such as `if/else` statements and `while-loop` exits) present a challenge: their target addresses are unknown at emit time because the respective block bodies have not yet been compiled. CVM++ resolves this using backpatching:

- `emitJump()` writes `OP_JUMP_IF_FALSE` followed by a `0xFFFF` placeholder, recording its index.
- The compiler proceeds to compile the AST block body normally.
- `patchJump()` calculates the exact byte distance and overwrites the `0xFFFF` placeholder.

For iterative 'while' loops, the backward jump to the loop condition utilizes a signed 16-bit offset (cast to `int16_t`), seamlessly instructing the Instruction Pointer to rewind the execution stream.

10. Virtual Machine (Execution Engine)

The VM is a highly efficient stack-based execution engine composed of three core components:

- Instruction Pointer (IP): Tracks the absolute execution position in the bytecode array.
- Value Stack: A `std::vector<int64_t>` facilitating all Last-In-First-Out computations.
- Global Memory: A `std::unordered_map` mapping variable names to their stored values.

The execution loop decodes and processes one opcode per iteration via an extensive switch statement. Runtime invariants (e.g., division by zero, stack underflow, undefined variables) are strictly verified, throwing `std::runtime_error` alongside the active IP state to aid debugging.

11. End-to-End Example

The following illustrates the compilation and execution of the Fibonacci sequence:

```
// Source (fibonacci.cvm)
let a = 0;
let b = 1;
let n = 10;
let i = 0;

while (i < n) {
    print a;
    let temp = a + b;
    a = b;
    b = temp;
    i = i + 1;
}
```

```
// Compiled Bytecode (Excerpt)
0000  OP_CONSTANT      0      (0)      | Push 0
0002  OP_SET_GLOBAL    0      (a)      | Store in 'a'
0004  OP_CONSTANT      1      (1)      | Push 1
0006  OP_SET_GLOBAL    1      (b)      | Store in 'b'
...   [loop condition evaluates]    |
0021  OP_JUMP_IF_FALSE 37          | Jump if condition fails
...   [loop body executes]          |
0058  OP_JUMP          [to 0016]    | Loop back to condition
0061  OP_HALT                          | Terminate VM
```

12. Validation and Test Suite

To ensure interpreter correctness, 13 dedicated .cvm test scripts validate the language's capabilities. All scripts have been verified to compile and execute without regression.

Test Script	Features Validated
arithmetic.cvm	+, -, *, / operator precedence
variables.cvm	let declarations, reassignment
booleans.cvm	true, false, ==, !=, ! operators
comparison.cvm	<, >, <=, >= relational ops
if_else.cvm	Simple and nested if/else branches
while_loop.cvm	Counting loop, sum accumulation
fibonacci.cvm	Iterative Fibonacci sequence calculation
nested_loops.cvm	Nested while loop constructs
control_flow.cvm	Combined branching and looping
input_output.cvm	print and input keyword capability
calculator.cvm	Variable-based mathematical expressions
edge_cases.cvm	Division by zero, runtime errors

13. Conclusion

CVM++ successfully implements a complete, functioning programming language ecosystem entirely from scratch. The four-stage pipeline (Lexer -> Parser -> Compiler -> VM) cleanly separates concerns and perfectly mirrors industry-standard compiler architectures. The project effectively demonstrates a practical, working understanding of lexical analysis, recursive descent parsing, AST construction, bytecode compilation with forward backpatching, and stack-based virtual machine execution.

The included REPL, file-runner interfaces, and comprehensive debug logging options ensure the system is highly introspective - making it both a practical utility and an excellent educational demonstration of core compiler construction principles.